

Geo-Automation Framework

Technical Documentation & Codebase Handover Guide

Application under test	Geo Travel web app (gowithgeo.com)
Framework type	Selenium + pytest, Page Object Model, UI + REST API coverage
Prepared by	John · john@blueoceantechsupport.com
Date	September 1, 2026
Repository	geo-travel-automation

Contents

1. Purpose & Scope
2. Technology Stack
3. Architecture Overview
4. Repository Layout
5. Environment & Configuration
6. Core Framework Reference
7. UI Page Object Reference
8. API Client Reference
9. Running the Suite
10. CI/CD Integration
11. Known Issues & Technical Debt
12. Recommendations for the Incoming Maintainer

1. Purpose & Scope

The Geo-Automation Framework is the QA automation suite for the Geo Travel web application. It provides end-to-end UI coverage (via Selenium, driving a real browser through booking flows) and REST API coverage (via `requests`, exercising the retail and partner/B2B backends directly). Tests run locally, on demand, and on a daily CircleCI schedule, with results surfaced as HTML/Allure reports and Slack notifications.

This document accompanies an inline documentation pass across the framework's core classes and page objects (37 files). Every module, class, and method in that scope now carries a docstring describing its purpose, parameters, return value, and — where relevant — known limitations. This document summarizes that codebase at a level suitable for a new maintainer to get oriented without reading all ~19,000 lines directly, and consolidates every bug or fragile pattern surfaced while writing that documentation into one place (Section 11).

Documentation Scope

The inline documentation pass covered **core framework classes and page objects** — the architectural backbone a new engineer needs first. It intentionally did not touch `src/tests/*` (the ~19 test suite files) or `src/utils/*` (15 shared helper modules), which remain as-is.

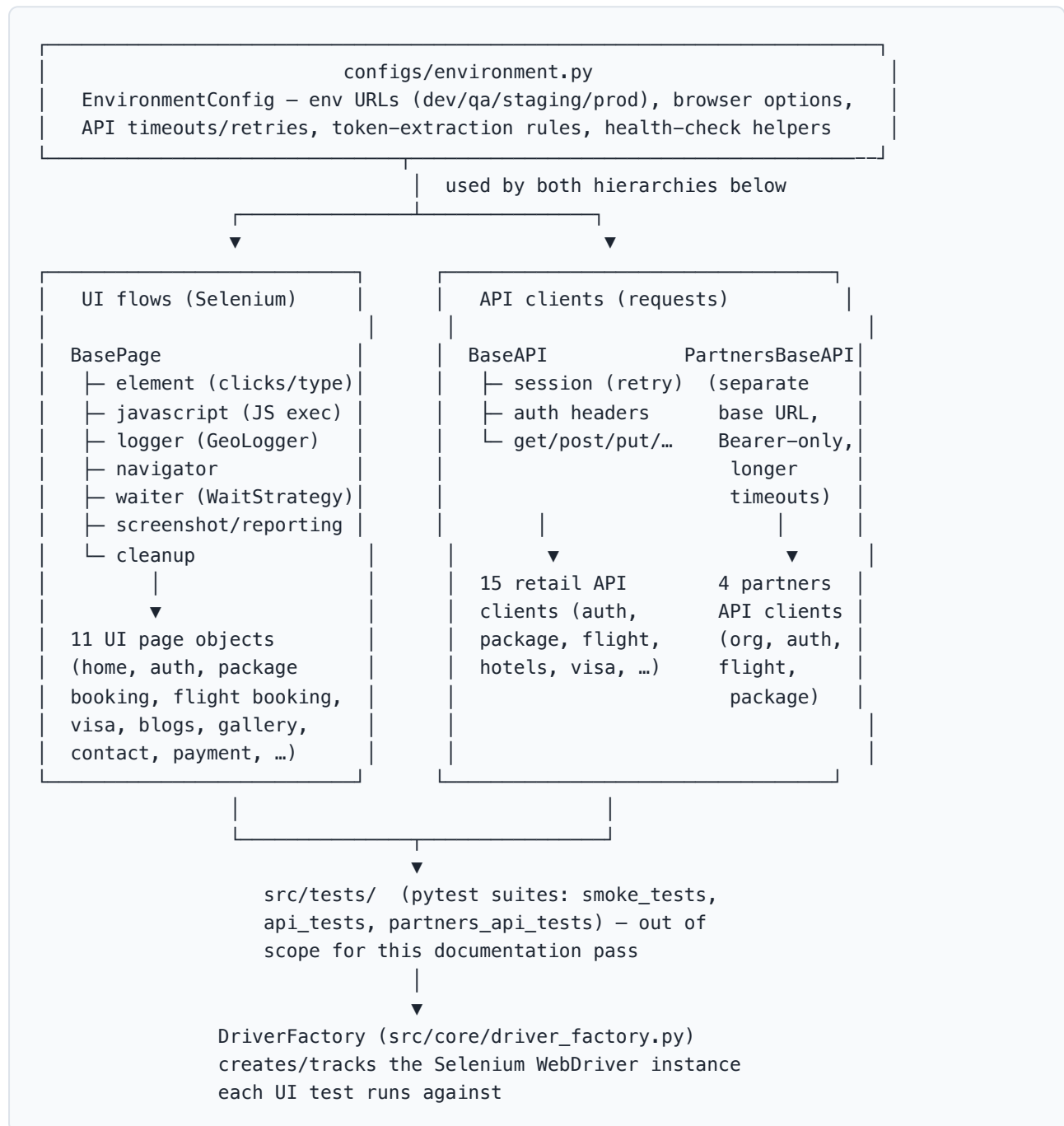
Area	Files	Status
Core framework (<code>src/core/</code> , <code>configs/</code>)	7	Documented
UI page objects (<code>src/pages/ui/</code>)	11	Documented
API clients (<code>src/pages/api/</code> , incl. <code>partners_api/</code>)	19	Documented
Test suites (<code>src/tests/</code>)	~19	Out of scope — unchanged
Shared utilities (<code>src/utils/</code>)	15	Out of scope — unchanged

2. Technology Stack

Concern	Tool / Library
Browser automation	Selenium WebDriver (Chrome, Firefox, Edge via <code>webdriver-manager</code>)
Test runner	pytest, with <code>pytest-html</code> for self-contained HTML reports and Allure for richer reporting
HTTP client (API tests)	<code>requests</code> , wrapped in a retrying session (<code>BaseAPI</code> / <code>PartnersBaseAPI</code>)
Config & secrets	<code>.env</code> file loaded via <code>python-dotenv</code> , read through <code>EnvironmentConfig</code>
CI/CD	CircleCI (scheduled daily smoke run) + GitHub Actions (<code>qa-tests.yml</code> , <code>auto-assign.yml</code>)
Notifications	Slack webhook (test result summaries, environment-down alerts)
Code quality tooling	black, flake8, isort, bandit, mypy-extensions (present in <code>requirements.txt</code>)

3. Architecture Overview

The framework follows the Page Object Model (POM): every user-facing flow or API resource is wrapped in a class that exposes intention-revealing methods (`login()` , `search_packages()`) instead of raw Selenium/HTTP calls. Two parallel class hierarchies exist — one for browser-driven UI flows, one for HTTP API clients — because the underlying I/O (WebDriver vs. `requests.Session`) is fundamentally different, but both follow the same shape: a shared base class owns cross-cutting concerns (logging, config, retries/waits), and each concrete class owns only the locators/endpoints and steps specific to its flow.



Key conventions worth knowing

- **Method chaining:** many UI page-object methods return `self` so steps can be chained (`navigate_to_packages().select_first_package()`).
- **Locators as class attributes:** every page object declares its Selenium locators as `(By.X, "...")` tuples at the top of the class, grouped by flow step under `# ===== SECTION =====` comments — locators are data, methods are behavior.
- **Cross-page coupling:** `package_booking_flow.py` imports and uses `PaymentPage` from `payment_flow.py` to verify checkout redirects. `flight_booking_flow.py` does *not* do this (see Section 11) — worth aligning if payment verification is extended.
- **Two independent API base classes:** `BaseAPI` and `PartnersBaseAPI` duplicate most of their retry/session logic rather than sharing an ancestor, and have already drifted slightly (Section 11) — a consolidation candidate.
- **Logging:** every class gets a `GeoLogger` instance scoped to its own class name (`self.logger = GeoLogger(self.__class__.__name__)`), used throughout for step-by-step traceability in reports.

4. Repository Layout

```
.
├── .circleci/config.yml      Scheduled daily smoke-test pipeline
├── .github/workflows/       qa-tests.yml, auto-assign.yml
├── .env                     Local secrets/config (not committed)
├── conftest.py              pytest fixtures shared across all suites
├── pytest.ini               pytest config: discovery rules, markers, HTML report
├── configs/
│   └── environment.py        EnvironmentConfig – env URLs, browser & API config
├── scripts/
│   ├── run_scheduled_tests.py CLI entry point used by CI (--suite smoke/api/...)
│   └── cleanup.py            Prunes old reports/screenshots
├── src/
│   ├── core/                Base classes: BasePage, BaseAPI, PartnersBaseAPI,
│   │                       DriverFactory, TestBase
│   ├── pages/
│   │   ├── ui/              11 Selenium page objects (documented)
│   │   └── api/              15 retail API clients + partners_api/ (4 more)
│   │                       (documented)
│   ├── tests/
│   │   ├── smoke_tests/     UI smoke suites, one per major flow
│   │   ├── api_tests/       Retail API test suites
│   │   └── partners_api_tests/ Partner/B2B API test suites
│   └── utils/                15 shared helpers: logger, waits, screenshots,
│                           JS execution, notifications, reporting, etc.
├── reports/                 Generated HTML/Allure test reports
└── logs/                    Run logs
```

5. Environment & Configuration

`configs/environment.py` defines `EnvironmentConfig`, a classmethod-based config hub (no instantiation needed) that switches behavior based on the `TEST_ENV` variable. It supports four named environments:

Environment	Retail base URL	Retail API
dev	retail.dev.gowithgeo.com	api.dev.gowithgeo.com
qa (default)	retail.qa.gowithgeo.com	api.qa.gowithgeo.com
staging	retail.stg.gowithgeo.com	api.stg.gowithgeo.com
production	www.gowithgeo.com	api.gowithgeo.com

Partner/B2B API URLs are configured separately per environment (sandbox URLs for dev/qa, distinct staging/production partner hosts).

Required `.env` variables

Names only — values are environment-specific secrets and are not reproduced here.

TEST_ENV	dev / qa / staging / production (default: qa)
BROWSER	chrome / firefox / edge (default: chrome)
HEADLESS	true/false
WINDOW_SIZE	e.g. 1920x1080
TIMEOUT	default Selenium wait timeout (seconds)
API_TIMEOUT / API_MAX_RETRIES	API client request tuning
API_TEST_EMAIL / API_TEST_PASSWORD	Retail API test account credentials
TEST_USER_EMAIL / TEST_USER_PASSWORD	UI login test account credentials
PARTNERS_VERIFIED_EMAIL / PARTNERS_VERIFIED_PASSWORD	Partners API verified test account
SLACK_WEBHOOK_URL	Slack notification target for test results
PROJECT_NAME / BRANCH / BUILD_ID / ENVIRONMENT	Metadata surfaced in reports/notifications

Note: `configs/environment.py::get_api_credentials()` reads `cls.API_TEST_EMAIL` / `cls.API_TEST_PASSWORD` as *class attributes*, but these are never actually defined on `EnvironmentConfig` — only read via `os.getenv()` inside `auth_api.py`. Calling this method raises `AttributeError`. See Section 11.

6. Core Framework Reference

File	Responsibility
<code>src/core/base_page.py</code>	<code>BasePage</code> — superclass for all Selenium UI page objects. Wires up <code>self.element</code> , <code>self.javascript</code> , <code>self.logger</code> , <code>self.navigator</code> , <code>self.pageinfo</code> , <code>self.reporting</code> , <code>self.screenshot</code> , <code>self.validator</code> , <code>self.waiter</code> , <code>self.cleanup</code> — every page object inherits this shared toolkit.
<code>src/core/base_api.py</code>	<code>BaseAPI</code> — superclass for retail API clients. Owns a <code>requests.Session</code> , manages auth headers/cookies (supports both cookie-based and Authorization-header token styles), and exposes <code>get/post/put/patch/delete()</code> , all routed through a retrying <code>_request()</code> (3 attempts).
<code>src/core/partners_base_api.py</code>	<code>PartnersBaseAPI</code> — parallel superclass for the 4 Partners/B2B API clients. Separate base URL, Bearer-token-only auth, longer timeouts, and dumps response bodies on failure for easier debugging. Duplicates rather than shares <code>BaseAPI</code> 's structure (see Section 11).
<code>src/core/driver_factory.py</code>	<code>DriverFactory</code> — creates and tracks Selenium WebDriver instances per the configured browser (chrome/firefox/edge), applying the options built by <code>EnvironmentConfig</code> .
<code>src/core/test_base.py</code>	<code>TestBase</code> — an optional pytest mixin adding setup/teardown timing, automatic failure screenshots, and smoke-test result reporting. Currently always reports "PASS" regardless of actual test outcome — see Section 11, this is the highest-priority fix in this document.
<code>src/core/__init__.py</code>	Re-export surface for the package (exposes <code>BasePage</code> , <code>DriverFactory</code> at the package level).
<code>configs/environment.py</code>	<code>EnvironmentConfig</code> — central config hub described in Section 5.

7. UI Page Object Reference

File	Flow covered
<code>home_page.py</code>	Homepage warm-up/load with retry logic, logo check, React/Next.js health check, and content-confidence validation that the right app loaded.
<code>auth_flow.py</code>	Login (email + password) and logout, including dashboard-indicator detection and toast-message capture.
<code>dashboard_page.py</code>	Post-login dashboard load and identity checks; wraps an <code>AuthFlow</code> instance to reuse its indicator locators.
<code>package_booking_flow.py</code>	Full package-booking journey: search (trip type, country, date) → select a price option → fill booking modal → log in → accept terms → choose a payment method → verify checkout destination (Flutterwave / Paystack / bank transfer). The most complete and heavily-used flow in the suite.
<code>flight_booking_flow.py</code>	One-way flight search (route currently hardcoded to Heathrow→Schiphol), result selection, passenger form, and payment-method click. Does not reuse <code>PaymentPage</code> or verify the checkout destination the way <code>package_booking_flow.py</code> does.
<code>payment_flow.py</code>	<code>PaymentPage</code> — triggers the Flutterwave hosted checkout and performs a minimal iframe-readiness check. Does not actually complete a card payment (card-entry locators/test data are defined but unused — Section 11).
<code>visa_enquiries_flow.py</code>	Visa application form: personal details, country/passport/visa-type dropdowns, travel date picker, submit/cancel.
<code>blogs_flow.py</code>	Blog listing, detail page, comment submission, and "More like this" related posts.
<code>travel_gallery_flow.py</code>	Gallery listing, tour detail navigation, hover-triggered image viewer modal.
<code>contact_flow.py</code>	Contact Support form submission, FAQ link, Privacy Statement link.
<code>__init__.py</code>	Empty package marker.

8. API Client Reference

Retail API ([src/pages/api/](#)) — subclass `BaseAPI`

File	Endpoint prefix	Covers
<code>auth_api.py</code>	<code>/api/auth</code>	Login (with dynamic token extraction), logout, token refresh.
<code>package_api.py</code>	<code>/api/package</code>	Listing, detail, booking, deals, a user's booked packages/analytics.
<code>flight_api.py</code>	<code>/api/flight</code>	Search request/results, booking, quotes, passenger email validation.
<code>flight_utils_api.py</code>	<code>/api/flight</code>	Reference-data lookups: airports, airlines.
<code>hotels_api.py</code>	<code>/api/hotels</code>	Search, cities, ratings, booking, priced offers.
<code>visa_enquiries_api.py</code>	<code>/api/visa</code>	Create enquiry, pay, verify payment, list/lookup applications.
<code>commercial_deals_api.py</code>	<code>/api/commercial</code>	Deal listing, booking, payment verification.
<code>transactions_api.py</code>	<code>/api/transactions</code>	User transaction list, insights, CSV export.
<code>users_api.py</code>	<code>/api/users/me</code>	Read/update the logged-in user's profile.
<code>notification_api.py</code>	<code>/api/notifications</code>	List, mark-read, delete notifications.
<code>newsletter_api.py</code>	<code>/api/newsletter</code>	Newsletter signup.
<code>price_api.py</code>	<code>/api/price/voucher</code>	Voucher lookup and apply.
<code>blogs_api.py</code>	<code>/api/blog</code>	Listing/fetching posts, reading/creating comments.
<code>events_api.py</code>	<code>/api/event</code>	List/fetch events.
<code>google_reviews_api.py</code>	<code>/api/google/reviews</code>	Surfaces Google reviews for the site.

Partners / B2B API ([src/pages/api/partners_api/](#)) — subclass `PartnersBaseAPI`

File	Endpoint prefix	Covers
<code>partners_auth_api.py</code>	<code>/api/auth</code>	Signup, login (with token extraction), email verification, password reset.
<code>organization_api.py</code>	<code>/api/org</code>	Organization profile, API key reset, usage reporting.
<code>partners_flight_api.py</code>	<code>/api/flight</code>	Search/book/list bookings, using per-call API-key auth.
<code>partners_package_api.py</code>	<code>/api/package</code>	List/book packages, countries, bookings, using per-call API-key auth.

9. Running the Suite

```
# Install dependencies
pip install -r requirements.txt

# Run everything
pytest

# Run a specific suite via the CI helper script
python scripts/run_scheduled_tests.py --suite smoke
python scripts/run_scheduled_tests.py --suite api
python scripts/run_scheduled_tests.py --suite partners_api
python scripts/run_scheduled_tests.py --suite api smoke partners_api
```

Test discovery, markers (`smoke` , `regression` , `sanity` , `api` , `partners_api` , `security` , `auth` , `cross_browser`), and HTML report generation are configured centrally in **pytest.ini**. Reports land in `reports/` , logs in `logs/` ; `scripts/cleanup.py` prunes artifacts older than a configurable retention window.

10. CI/CD Integration

- **CircleCI** (`.circleci/config.yml`) runs a daily scheduled smoke + API test job at 07:00 UTC (08:00 Nigeria time), storing screenshots as artifacts and cleaning up test artifacts older than 7 days afterward. A regression-test job exists in the config but is currently commented out.
- **GitHub Actions** (`.github/workflows/`) runs `qa-tests.yml` and `auto-assign.yml` (the latter auto-assigns reviewers/comments on issues and PRs).
- **Slack** notifications are sent for test run summaries and environment-down alerts, via `SLACK_WEBHOOK_URL`.

11. Known Issues & Technical Debt

Everything below was discovered incidentally while writing docstrings for the 37 files in scope — this pass was documentation-only, so **nothing here has been fixed**; each is also flagged with a `NOTE: / FIXME:` comment at its location in the code. They're consolidated here so the next maintainer has a prioritized starting point instead of discovering them one at a time.

SECURITY Secrets/tokens logged in plaintext

```
src/core/base_api.py - _request()
```

Every outgoing request logs the full headers dict at INFO level, including live `Authorization` and `Cookie` values — these end up in plaintext in logs/CI output for every retail API test run.

```
src/pages/api/partners_api/partners_flight_api.py - _get_flight_headers()
```

Logs the full headers dict at INFO level, including the `x-api-secret` value — a plaintext secret leak into test logs/CI output on every partners flight-API call.

RELIABILITY Bugs that undermine trust in test results

```
src/core/test_base.py - TestBase.teardown_method()
```

Reads pass/fail status from `self._test_outcome`, which no code anywhere in the repo ever sets (confirmed by repo-wide search). **Any test built on this mixin always reports "PASS" to smoke reporting, regardless of what actually happened.** Relatedly, `mark_test_failed()` sets `self._test_failed / self._error_message`, which `teardown_method()` never reads — so calling it currently has no effect either. This is the single highest-priority item in this list if any reporting relies on `TestBase`.

```
configs/environment.py - get_api_credentials()
```

References `cls.API_TEST_EMAIL / cls.API_TEST_PASSWORD` as class attributes that are never defined on `EnvironmentConfig` — calling this method always raises `AttributeError`.

```
src/pages/ui/package_booking_flow.py - is_price_option_available()
```

Iterates `self.PRICE_OPTIONS`, which doesn't exist on the class (only `PRICE_OPTION_BY_TYPE`, a dict keyed by option type, is defined) — raises `AttributeError` instead of returning a bool as documented.

```
src/pages/ui/dashboard_page.py - wait_for_dashboard_load()
```

Passes `AuthFlow.DASHBOARD_INDICATORS` (a list of locators) into helpers that expect a single `(By, value)` tuple — likely raises at runtime rather than checking each indicator in turn.

```
src/pages/ui/home_page.py - wait_for_homepage_load()
```

On the final failed-validation path (not the exception branch), raises `AssertionError(f"...: {e}")` where `e` is undefined in that scope — produces a `NameError` instead of the intended assertion message.

`src/pages/ui/visa_enquiries_flow.py` – `click_get_started()`

Falls through to an implicit `None` return after exhausting retries instead of raising or returning `self` — breaks any chained call relying on the usual `self` -return convention.

`src/pages/ui/travel_gallery_flow.py` – `verify_gallery_has_tours()`

Its `assert count > 0` is inside the method's own `try/except Exception` block, so a failed assertion is swallowed and logged rather than ever propagating to the caller/test.

`src/core/partners_base_api.py` – `_request_with_retry()`

`ReadTimeout` appears in two different `except` tuples (the second is unreachable dead code). Separately, the `ConnectionError / ChunkedEncodingError` branch never raises or returns on the final retry attempt, so exhausting retries there silently returns `None` — the original exception is lost, and `_request()` converts this into a generic, less-informative `ValueError`.

FUNCTIONAL GAP **Methods that don't do what their name/docstring implies**

`src/pages/ui/payment_flow.py` – `complete_payment_flow()`

`CARD_NUMBER_INPUT`, `EXPIRY_DATE_INPUT`, `CVV_INPUT`, `PAY_BUTTON`, and `TEST_CARD_DETAILS` are all defined but never used anywhere in the file. The method only confirms the Flutterwave iframe loaded with some inputs present — it never actually fills in card details or submits a payment, despite the name.

`src/pages/ui/flight_booking_flow.py`

Doesn't use `PaymentPage` at all (unlike `package_booking_flow.py`); its `select_payment_method` never verifies the flow actually reaches the Flutterwave checkout URL. Several passenger-form locators (passport, date of birth, country of origin) are defined but never filled in by `fill_passenger_information`.

`src/pages/ui/visa_enquiries_flow.py` – `fill_personal_details()`

Accepts `first_name` / `last_name` / `email` / `phone` parameters but ignores all of them, always filling in hardcoded test values instead.

`src/pages/ui/visa_enquiries_flow.py` – `select_travel_date()`

Accepts/computes a `date_text` value but never uses it — always clicks the first enabled day-of-month button greater than 10, regardless of the date requested.

`src/pages/ui/contact_flow.py` – `submit_form()`

Accepts a `max_retries` parameter but implements no retry logic using it.

`src/pages/api/hotels_api.py` – `get_hotel_offers()`

Docstring/comment says the endpoint is `/api/hotels/offerReq`, but the code actually calls `/api/hotels/offer` — likely a stale comment from a prior endpoint rename.

```
src/pages/api/flight_api.py – get_search_results(), get_booked_flights(), get_quote()
```

Build query strings via raw f-string interpolation directly into the endpoint path, instead of passing a `params=` dict as sibling methods/files do — values aren't URL-encoded, so a value containing spaces or special characters could produce a malformed request.

```
src/pages/api/visa_enquiries_api.py – get_visa_application_by_id()
```

Passes the ID as a query parameter (`?visa_application_id=`) rather than a path segment, inconsistent with the `/ {id}` pattern used by equivalent lookups elsewhere in the same file and sibling API clients.

```
src/pages/api/partners_api/organization_api.py – reset_api_keys()
```

Uses **GET** for what sounds like a mutating action (rotating/invalidating API keys) — likely should be **POST**.

CODE QUALITY **Dead code & drift**

```
src/pages/api/package_api.py – book_package()
```

Defined **twice** in `PackageAPI` with identical bodies — the second silently shadows the first. Harmless today, but a future edit to only one copy would create real behavioral divergence.

```
src/core/base_api.py – _request()
```

A safety check guards on `self._debug_token_source`, an attribute that's never set anywhere (`set_auth_token()` sets `self.token_source` instead) — this check is permanently dead code.

```
src/pages/api/hotels_api.py – get_headers()
```

Builds its own headers dict independently of `self.headers` (set up in `BaseAPI.__init__`), using the key `"Client-Type"` instead of `"X-Client-Type"`. Not used by any live request path — only referenced by a commented-out legacy method.

```
src/pages/ui/blogs_flow.py – navigate_to_blogs()
```

Has a dead `except TimeoutError` clause that can never execute, since it's already caught by a preceding `except Exception`.

```
src/pages/ui/dashboard_page.py – LOGOUT_BUTTON
```

Includes `button:contains('Logout')` — a jQuery-only pseudo-class, invalid in native CSS — inside a comma-separated selector, which likely breaks the whole selector. Also unused by any method in the class.

```
configs/environment.py – wait_for_api_environment()
```

Accepts a `check_interval` parameter that's never used in the body; the wait time is hardcoded as `3 * attempt` instead.

```
configs/environment.py – override_token_extraction_config()
```

Silently no-ops if the given `environment` isn't already a known key, dropping `config_updates` without any error or warning.

`src/core/partners_base_api.py`

Duplicates most of `BaseAPI` 's structure independently rather than sharing a common ancestor; the two retry implementations have already drifted from each other (see the reliability item above).

Stale header comments (cosmetic, not fixed – out of scope)

Several files' leading `# path/to/file.py` comments no longer match their actual filenames/casing, e.g. `transactions_api.py` says `transaction_api.py`, `users_api.py` says `user_api.py`, and the `partners_api/` files say `Partners_api/` with a capital P — leftovers from a prior rename.

12. Recommendations for the Incoming Maintainer

1. **Fix `TestBase.teardown_method()` first.** If any test suite relies on this mixin for pass/fail reporting, results have been silently wrong — this should be verified and fixed before trusting any historical "all green" report that used it.
2. **Stop logging live secrets.** Redact or remove the header-dump debug logging in `base_api.py` and `partners_flight_api.py` before this reaches any log aggregation system outside CI's ephemeral storage.
3. **Triage the "functional gap" methods** (Section 11) — several tests may currently be passing without actually exercising the behavior their names promise (e.g. payment completion, passenger-detail filling). Decide whether each was intentionally simplified or is genuinely incomplete.
4. **Consider consolidating `BaseAPI` and `PartnersBaseAPI`** behind a shared retry/session base to stop the two retry implementations from drifting further apart.
5. **Extend this documentation pass to `src/tests/` and `src/utils/`** when time allows — they were intentionally out of scope here to keep this first pass focused on the architectural backbone.

All findings above are also recorded as `NOTE: / FIXME:` comments at their exact location in the source, so they'll surface again naturally to anyone reading or modifying that code.